Dashboard & Aggregation — The Live Path vs. Two Layers of Dead Strategy Code

InstantMudra Enterprises Admin Portal

⚠ CRITICAL — DashboardController has zero access control

@RequestMapping("/api/dashboard") — 4 endpoints, none call HierarchyAccessService or HierarchyAccessValidator

`POST /get · GET /wallet-balance/{hierarchyId} · GET /profile/{hierarchyId} · POST /data`

Every method takes hierarchyId directly (path variable or request body) and passes it straight into DashboardServiceImpl with no check that it belongs to, or is a descendant of, the caller's own identity.

A classic IDOR — any authenticated user, at any level, can view any OTHER hierarchy's wallet balance and profile

by ID alone — no token spoofing required. This is distinct from, and compounds, the Module 4 identity-spoofing bug.

⚠ Confirmed bug: Enterprises profile lookup is broken

DashboardServiceImpl.getUserProfile(hierarchyId) switches on commonMethods.extractHierarchyType(hierarchyId):

```
case "EN" -> enterprisesRepository.findByEnterpriseId(...) // real Enterprises IDs extract to type "ET", never "EN"
default -> throw new RuntimeException("Unsupported hierarchy type: " + type)
```

Per Module 2/1, HierarchyMaster seeds the code "ET" for Enterprises — not "EN". Every real Enterprises hierarchyId falls through to default and throws "Unsupported hierarchy type: ET" — this endpoint is broken for the entire ET level.

Two dead layers of the intended strategy pattern

Layer 1 — DashboardStrategy (buildDashboard)

- ADDashboardStrategy — stub, returns Map.of() unconditionally
- RtDashboardStrategy — full, real implementation (150 lines): balances, last-5 transactions, aggregation cards
- SdDashboardStrategy — full, real implementation (129 lines), same shape for SuperDistributor

Confirmed by search: referenced by NOTHING outside their own 4 files. No controller, no service ever selects one.

Two real, working implementations sit completely unused.

Layer 2 — HierarchyAggregationStrategy

Exactly ONE implementation: HierarchyAggregationStrategyImpl — a plain @Service, no name or @Qualifier given.

`Map<String, HierarchyAggregationStrategy> strategies` is keyed by Spring bean NAME, not hierarchy code — so strategies.get("RT") / .get("SD") / .get(anything)

always returns null → getStrategy() always throws IllegalArgumentException, for every type, no exceptions.

If Layer 1 were ever wired up, it would break here immediately — RtDashboardStrategy calls getStrategy("RT") directly.

Even setting the keying bug aside: HierarchyAggregationStrategyImpl.getTransactionVolume() and .getServicePieData()/.getLast4MonthsGraphData() are themselves stubs returning null/empty.

The live path — DashboardServiceImpl (2,416 lines)

DashboardController → DashboardService → DashboardServiceImpl (the actual, working dashboard)

Switches on hierarchy type internally

commonMethods.extractHierarchyType() called 4× inside this one class — its own type-branching, independent of both dead strategy layers above.

Its own recursive subtree walk

userDetailsRepository.findByParentAccount(), walked recursively — rather than the AccessHierarchyPath closure table built for exactly this in Module 2.

Live path / standard structure Flagged bug / dead code Critical — confirmed access-control gap or always-throws bug